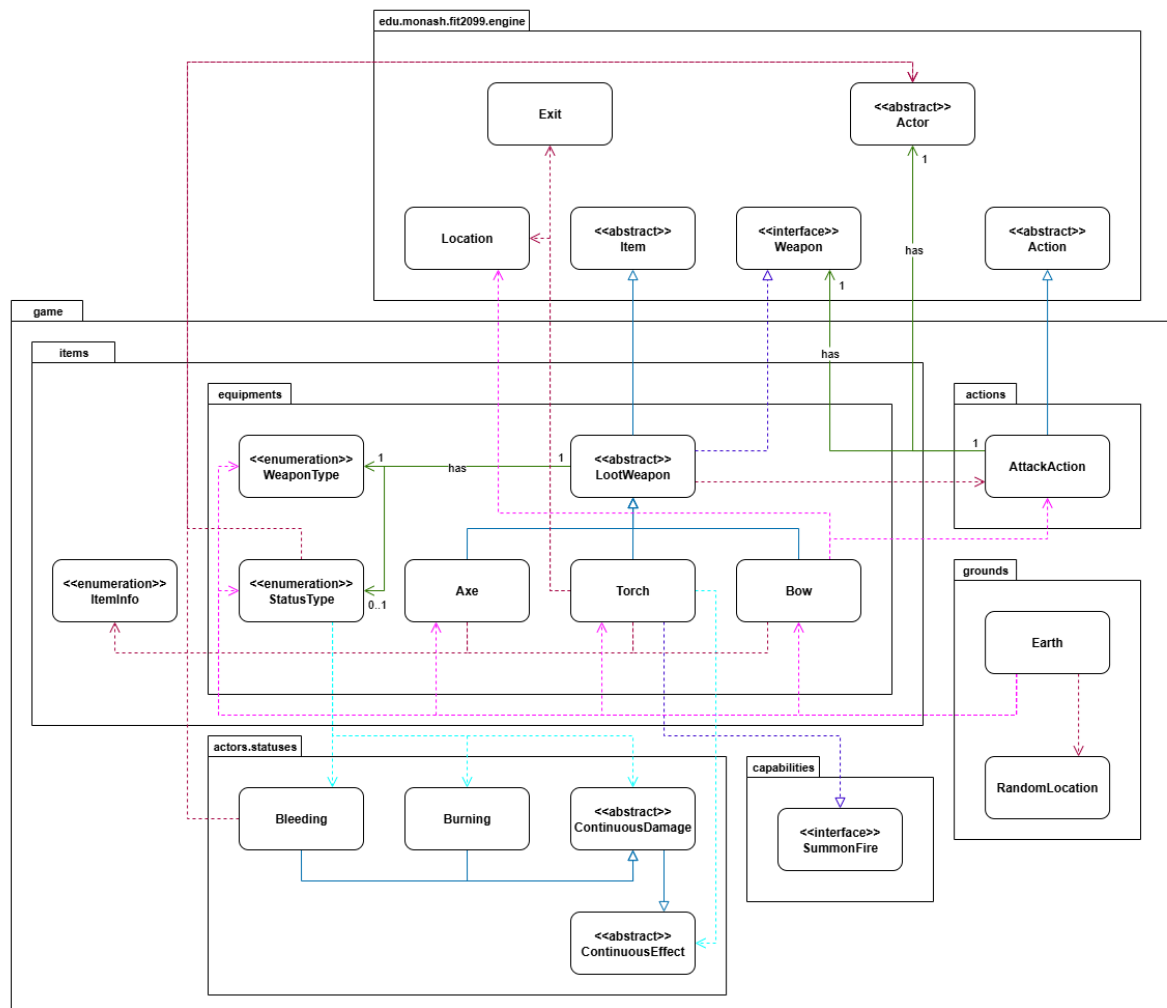


Design Rationale Requirement 3

Design Diagram



Design 1: Create `WeaponType` and `StatusType` enumerations to encapsulate weapon configuration values and effect values. Thus, the `LootWeapon` abstract class and its subclasses don't need to define common constants across classes.

Pros	Cons
Centralise configurations; all weapon stats and status effect values live in one place to avoid magic numbers being scattered across classes.	Lose some flexibility; values in the enum are fixed, if new stats want to be used, a new enum constant must be added.
Reduce primitive obsession smell; primitive data types like <code>int</code> and <code>String</code> are encapsulated in meaningful objects.	Existence of connascence of meaning; if the original enum values are being changed, other parts of code may break, like the conditional check that relies on enum values.
Consistency; modify weapon stats or status effect values in enum instead of in each weapon class and it will update everywhere automatically.	
Reduce long parameter lists; related values are encapsulated in enum classes to reduce the risk of connascence of position.	
When creating a weapon object, it can customise different weapon stats and status effects without touching the weapon class itself.	

Design 2: Instead of creating `WeaponType` and `StatusType` enumeration, put constants that represent the weapon stats and effect values.

Pros	Cons
Simplicity; all values are visible in each weapon class.	Code duplication; a lot of same constants but different values across multiple weapon classes.
	Increase the risk of connascence of position; without enum to encapsulate those values, each weapon class may have long parameter lists and easy to put wrong order when creating an object.
	Hard to maintain; if a weapon damage change, we must go to that class to adjust.

In requirement 3, the expected existing classes have LootWeapon, Axe, Torch, Bow, ItemInfo, WeaponType, StatusType, AttackAction, Bleeding, and SummonFire. The modified classes have ContinuousDamage, Burning, and Earth. The whole of requirement 3 is mainly to adopt Design 1.

Firstly, the Axe, Torch, and Bow classes inherit from the LootWeapon superclass. Therefore, common attack behaviour, `attack()`, lives in the base class, and it implements customisation of `hit()` method in each subclass to represent different attack behaviours, such as causing ContinuousDamage like using an Axe may cause the target to bleed and Torch causes the target to burn. Thus, this follows the Liskov Substitution Principle because Axe, Torch, and Bow can replace LootWeapon safely. For the Bow, the player holding the Bow uses himself as a 3-radius range. Once other actors enter the range, the Player can use the bow to shoot them.

Since the LootWeapon abstract class implements the Weapon interface from the engine, any classes that inherit from the LootWeapon class will also be a type of weapon. Therefore, each subclass of the LootWeapon class or the LootWeapon class itself doesn't directly associate with the AttackAction itself, instead the AttackAction class will accept any item as long as it is a type of Weapon, which follows the Dependency Inversion Principle because the concrete classes don't depend on each other but on an abstract class between them. This also follows the Open/Closed Principle because creating more new weapon classes doesn't require modifying the AttackAction class.

The creation of WeaponType and StatusType enumeration will help us to reduce some connascences. Firstly, without WeaponType, we may have many of the same constants that are scattered across the weapon classes like damage, hit rate, and attack verb but in different values. The WeaponType enum centralises these configurations and avoids the existence of magic numbers (connascence of meaning) and significantly reduces the connascence of value and shotgun surgery because we only change the values in one place and other parts of the code will be automatically updated, and this also applies to StatusType. Same as the ItemInfo enum, since LootWeapon is a type of Item, we use ItemInfo to encapsulate item information to avoid magic strings in the constructor, like `super("Axe", 'p', true)`; instead, we do `super(ItemInfo.AXE.getName(), ItemInfo.AXE.getDisplayChar(), ...)`. Not only that, but without enum, the constructor of the weapon class may also be too long, like `"public Axe(int damage, int hitRate, String verb...)"`, which increases the risk of connascence of position. On the other hand, enum centralises these attributes to avoid long parameter lists also and help us to pass a few things only, like `"public Axe(WeaponType, StatusType)"`. Thus, enum also removes the primitive obsession because those primitive data types are encapsulated in a class, so when we want to use it, we just pass enum instead of many primitive data types.

Lastly, the Earth class instantiates every weapon object and acts like a dependency injector because some weapons may have their own status. The weapon is randomly scattered on a location within the map. With the help of enums, we can inject different statuses in the Earth class without having to go to that class to change to different statuses. Therefore, the enumeration of WeaponType and StatusType is just like a factory that produces various

components, and we assemble them in the Earth class. In the future, if a weapon can have many statuses, we can inject a list of statuses that contains different statuses, such as `status.addAll(List.of(StatusType.BLEEDING, StatusType.BURNING, ...))`.